

Attestor

A cross-chain loan repayment history you cannot lie to — built on USC.

BUIDL CTC 2026 FALL · ETHEREUM SEPOLIA → CREDITCOIN CC3 TESTNET ·
GITHUB.COM/RICHARDREKI/ATTESTOR

THE PROBLEM

Creditcoin exists to make credit history verifiable. But the money that builds that history — stablecoin loans and repayments — lives on Ethereum. Bridging the *record* across chains usually means trusting a relayer's word that a repayment happened. A credit history is worthless if it can be forged.

THE SOLUTION

A borrower repays on Ethereum; an autonomous agent posts it to a credit history on Creditcoin — but the record is written **only** if an Attestcoin proof shows the repayment really happened. The agent proposes; the on-chain book disposes. **Neither the agent nor we can write a false entry.**

AAVE V3 · SEPOLIA

A stranger repays a debt. **Not our contract, not our borrower** — we cannot cause, populate or alter the `Repay` event.

OFF-CHAIN AGENT

Watches, applies a risk gate, waits ~8 min for attestation, fetches the proof, proposes a posting.

CREDITCOIN CC3

`AaveLoanBook` verifies the proof on the USC BlockProver precompile, re-derives the event from the attested receipt, writes the history.

Remove USC and there is no product: the book holds no trusted input of its own — no owner-submitted number, no oracle, no signed report. Every write is downstream of one precompile call, `0x...0FD2 . verify(...)`, returning without reverting.

Why the source matters more than the checks. A history you issue to yourself proves only that you can issue history to yourself. `AttestedLoanBook` — same precompile, same checks, but proving repayments to a contract we wrote — is kept alongside as the control. The contrast is the argument.

THE SECURITY MODEL — USC PROVES LESS THAN IT APPEARS

It proves a transaction was **included in a confirmed block** — not that it *succeeded*, touched the contract you care about, or was sent by who it claims. The book enforces, on-chain, every property the proof does not:

CHECK	WITHOUT IT
<code>status == 1</code>	a reverted repayment posts as if it paid
<code>chainId (resolved)</code>	a valid proof from the wrong chain is accepted
<code>emitter + topic, as a pair</code>	anyone deploys a contract emitting Aave's event and is believed
<code>borrower from the event</code>	credit is assigned to whoever relays the proof
<code>repayer == borrower</code>	one funded account mints standing for any address it likes
<code>freshness, in source blocks</code>	an attested repayment is postable forever
<code>consumed per Log</code>	a transaction repaying several debts records only the first

Three of those are forced by reading a protocol we did not write. Aave's `repay` takes `amount = uint256 max` to mean "all of it", so the calldata does not say how much was repaid — only the event does. One transaction can repay several debts; a routed repayment on this pool was observed at log index 500. And a USC attestation carries no timestamp anywhere, so age is `latestAttestedHeight - provenHeight` and cannot honestly be quoted in seconds. **51 contract tests; 28 reject a forged or invalid input.**

WHAT WE FOUND THAT THE DOCS DON'T SAY – SEVEN UNDOCUMENTED PROTOCOL FACTS

- 1 **The precompile reverts on a bad proof; it does not return `false`** as the SDK documents — so `if (!verified)` is dead code.
- 2 **~8-minute latency is a real constraint:** a 32-block reorg window named only in an error string, plus ~39 blocks of attestation lag. We surface it, not hide it.
- 3 **`chainKey` is environment-scoped** — key 1 is Sepolia on testnet, Ethereum mainnet on mainnet. Bind to `chainId`, or a promotion silently changes trust.
- 4 **SDK and chain names differ:** the function is `verify` on-chain, not the SDK's `verifySingle`; `ChainInfo` is `snake_case` on-chain.
- 5 **The proof envelope is `(uint8,bytes[])`, not `bytes[]`** — and the SDK's own docs say the wrong one. A consumer written from the docs decodes nothing.
- 6 **The precompiles can't be fork-tested, and the failure is silent** — a call to the codeless address succeeds and returns nothing, passing the wrong assertion.
- 7 **`get_chain_by_key` returns one tuple, not two values** — a bug we shipped; every mocked test passed, it reverted every time on-chain. It's why we built a live-check layer.

Findings 5 and 7 make a documentation-following consumer fail outright. They are our ecosystem contribution as much as the product is.

LIVE & VERIFIABLE – NO KEY, NO GAS

reproduce	<code>node spike.mjs</code>	AaveLoanBook <code>0xc3762daB...61ea</code> (CC3)
tests	<code>forge test</code> – 51 pass	Aave V3 Pool <code>0x6Ae43d32...8951</code> (Sepolia) – not ours
live	<code>node tools/live-check.mjs</code>	repay (Sep) <code>0x2f8901c4...</code> post (CC3) <code>0x7fb71b18...</code>
agent	20 tests, tsc clean	CreditLine <code>0xA595C959...Bbb2</code> (CC3) – reads the Aave book
		control AttestedLoanBook <code>0xe31906a2...5e9</code> ·
		CreditLine <code>0xC45f8594...</code>
		BlockProver <code>0x...0FD2</code> ChainInfo <code>0x...0FD3</code>

A stranger's repayment is now credit history on Creditcoin. `0x2C56b94f...6A88` repaid **25 USDC to Aave V3** on Sepolia (`0x2f8901c4...`, block 11,605,409). We did not send it, did not fund it, and learned it existed by reading the chain. Its proof was posted to `AaveLoanBook` (`0x7fb71b18...`, 221,790 gas), which now reads `totalRepaid 25000000` for that borrower. A `CreditLine` bound to that book (`0xA595C959...Bbb2`) then reports a **credit limit of 25000000** for them, against **0** for an address with no Aave history — both public view calls anyone can make. We cannot draw that loan and do not pretend to: `borrow()` `credits msg.sender` and the key is theirs. The history belongs to the borrower, so the money does too. Run `tools/post-aave.mjs` again and it picks a *different* repayment, because people keep making them; a pinned fixture would go stale and quietly stop meaning anything.

The control, deliberately kept: the same machinery against a source contract of our own closed the full loop earlier — repay 250 mUSD (`0x49592b0c...`) → post (`0x0f3d4ca0...`) → a **real 125 mUSD undercollateralised loan** drawn against the attested history (`0xb62ffcff...`). Every check passes on both paths. Only one is evidence about anybody's creditworthiness, and it is not the one where we wrote the borrower, the token and the contract it paid.

Next: the agent watching Aave continuously rather than one repayment at a time · more source protocols, since a market is configuration once the event layout is known · batch proofs (the precompile takes up to 10, measured) · CC3 mainnet against Ethereum Aave, where the same `chainKey` means a different chain and the resolution check earns its keep.